Internal Spectral Utilities Specification

Correlation Transforms, Discrete Bin Measures, and Atom-Blocked Spectrum Planning

mdstats

2026-07-15

Contents

Purpose and status

This document specifies the private module

`mdstats/analysis/_spectral.py`

The module transforms uniformly sampled, positive-lag correlation arrays into one-sided spectral densities. It is a numerical foundation for `velocity_spectrum.py` and later current-, stress-, and other correlation-spectrum modules.

The module is not public API. Public modules own physical selection, normalization, units, interpretation, and result objects.

Separation from `_fft.py`

The existing `_fft.py` computes linear positive-lag correlations from sampled signals:

```
sampled signal -> linear correlation
```

The new `_spectral.py` performs the distinct operation

`stored positive-lag correlation -> spectral density`

The operations have different padding and scaling contracts. In particular, `_fft.linear_fft_length()` pads a signal to prevent circular correlation, whereas `resolve_spectrum_fft_length()` allocates one complete two-sided correlation plus optional spectral zero padding.

Provenance

Borrowed theory

For a stationary process, the power spectral density is the Fourier transform of its autocorrelation. This is the Wiener-Khinchin relation [1, 2].

Window leakage and resolution trade-offs are standard harmonic-analysis machinery; the implementation cites Harris [3]. FFT execution is supplied by SciPy [4].

mdstats design

The following are package-specific choices:

- positive lag is always axis zero;
- tensor negative lags are reconstructed by transposition;
- built-in lag tapers are centered at lag zero and satisfy $w_0 = 1$;
- a one-sided density is returned;
- zero padding is described only as frequency-grid refinement;
- every helper validates shapes, finiteness, and exact lower bounds.

Mathematical convention

Let $C[k]$ be a real correlation stored for

$$k = 0, 1, ..., L - 1,$$

with sample spacing Δt.

For a scalar or a diagonal self correlation,

$$C[-k] = C[k].$$

For a Cartesian tensor,

$$C_{\alpha\beta}[-k] = C_{\beta\alpha}[k].$$

A real periodic work array $g[n]$ of length N_{FFT} stores the measured positive lags at the beginning, negative lags at the end, and zeros between them. The minimum collision-free length is

$$N_{\mathrm{FFT}} \geq 2L - 1.$$

The nonnegative-frequency transform is

$$S(f_m) \approx \Delta t \, \mathrm{RFFT}[g]_m.$$

For a one-sided density, positive interior bins are doubled. DC remains unchanged. The Nyquist bin remains unchanged when the FFT length is even.

Type aliases

```
FloatArray = NDArray[np.float64]
ComplexArray = NDArray[np.complex128]
LagWindowInput = str | tuple[str, float] | ArrayLike
```

Atom-spectrum planning data structure

```
@dataclass(frozen=True, slots=True)
class AtomSpectrumPlan:
```

```
    n_fft: int
    n_frequency: int
    atom_block_size: int
    estimated_work_bytes: int
```

This structure was introduced in `mdstats 0.19.6a0` as roadmap unit N3.1 and is consumed by VS2 in `mdstats 0.19.7a0`. It supplies bounded-memory atom blocks to the VS2 direct Welch estimator. It does not execute an FFT or choose segment/window parameters.

Function specifications

`make_atom_spectrum_plan`

```
def make_atom_spectrum_plan(
    n_atoms: int,
    segment_length: int,
    n_fft: int,
    *,
    atom_block_size: int | None,
    memory_target_bytes: int,
) -> AtomSpectrumPlan:
    ...
```

Provenance

This planner adapts the existing mdstats `_fft.make_atom_fft_plan` architecture to direct spectral work arrays. It introduces no borrowed external algorithm. The byte model follows NumPy `float64`/`complex128` storage sizes and the standard real-FFT output length $F = \lfloor N_{\mathrm{FFT}}/2 \rfloor + 1$. SciPy will later execute the transform, but this planning function does not call SciPy.

Inputs and constraints

- `n_atoms`, `segment_length`, `n_fft`, and `memory_target_bytes` are positive integers; Boolean values are rejected.
- `n_fft >= segment_length`.
- `atom_block_size` is either `None` or a positive integer.
- an explicit block size larger than `n_atoms` is clamped to `n_atoms`.
- `memory_target_bytes` is a soft planning target, not a promise about opaque FFT-backend allocations.

Memory model

For one atom, the conservative Python-visible work estimate includes:

1. one three-component real segment of length L;
2. one three-component padded real work array of length N_{FFT};
3. one three-component complex RFFT of length F;
4. one three-component real diagonal-periodogram scratch array of length F.

With $b_r = 8$ bytes and $b_c = 16$ bytes,

$$B_{\mathrm{atom}} = 3Lb_r + 3N_{\mathrm{FFT}}b_r + 3Fb_c + 3Fb_r.$$

For block size B,

$$B_{\mathrm{work}} = BB_{\mathrm{atom}}.$$

When no explicit block is supplied,

$$B = \max\left(1, \min\left(N_{\mathrm{atoms}}, \left\lfloor \frac{B_{\mathrm{target}}}{B_{\mathrm{atom}}} \right\rfloor\right)\right).$$

A target smaller than one atom still yields block size one so the estimator can run. `estimated_work_bytes` may therefore exceed the requested target in that case. Estimator-wide accumulators and implementation-private FFT workspace are excluded and must be documented by VS2.

Required tests

- exact frequency count for even and odd FFT lengths;
- explicit block sizes and clamping;
- automatic block selection at exact byte boundaries;
- one-atom fallback for a tiny target;
- exact `estimated_work_bytes`;
- Boolean, nonpositive, and undersized-FFT rejection.

`resolve_spectrum_fft_length`

```
def resolve_spectrum_fft_length(
    n_positive_lags: int,
    *,
    zero_pad_to: int | None,
) -> int:
    ...
```

Inputs

- `n_positive_lags` is the number L of stored nonnegative lags and must be a positive integer.
- `zero_pad_to` is an optional positive-integer lower bound on the final work length.

Algorithm

1. Compute the reconstruction lower bound $2L - 1$.
2. If supplied, take the maximum with `zero_pad_to`.
3. Return `scipy.fft.next_fast_len(lower_bound)`.

Edge cases

- Boolean values are rejected even though Python treats `bool` as an integer.
- A padding request smaller than $2L - 1$ does not shrink the required work array.
- Padding changes the frequency spacing, not the physical resolution imposed by the measured correlation duration.

`resolve_lag_window`

```
def resolve_lag_window(
    window: LagWindowInput | None,
    n_lags: int,
) -> tuple[FloatArray, dict[str, Any]]:
    ...
```

Supported inputs

- `None`: rectangular truncation;
- `"half_hann"`;
- `("half_tukey", alpha)` with $0 \le \alpha \le 1$;
- a custom finite one-dimensional array of length `n_lags` satisfying $w_0 = 1$.

The half-Hann taper is

$$w_k = \frac{1}{2}\left[1 + \cos\left(\frac{\pi k}{L - 1}\right)\right].$$

It begins at one and reaches zero at the last measured lag. Applying an ordinary full Hann array directly would incorrectly erase $C(0)$.

For the half-Tukey taper, `alpha` is the fraction of the positive-lag interval occupied by the terminal cosine taper. `alpha=0` gives a rectangular window; `alpha=1` gives the half-Hann window.

The returned metadata records the resolved name, kind, parameter, and custom values when applicable.

`reconstruct_two_sided_correlation`

```
def reconstruct_two_sided_correlation(
    positive_lag: ArrayLike,
    *,
    n_fft: int,
    tensor_axes: tuple[int, int] | None = None,
) -> FloatArray:
    ...
```

Input layout

- lag is axis zero;
- all values are real and finite;
- `n_fft >= 2 * n_lags - 1`;
- tensor axes, when supplied, are distinct equal-length axes and exclude the lag axis.

Algorithm

For scalar-like input:

```
work[0:L]             = C[0:L]
work[N-L+1:N]         = reverse(C[1:L])
unused middle samples = 0
```

For tensor input, transpose the two tensor axes before reversing the positive-lag block. This enforces

$$C_{\alpha\beta}(-t) = C_{\beta\alpha}(t).$$

Motive

A full tensor at positive lag is generally nonsymmetric. A scalar even extension would discard the antisymmetric time-ordering information and force an incorrect real symmetric cross spectrum.

`one_sided_density_scale`

```
def one_sided_density_scale(n_fft: int) -> FloatArray:
    ...
```

For odd N:

```
[1, 2, 2, ..., 2]
```

For even N:

```
[1, 2, 2, ..., 2, 1]
```

The final undoubled value for even N is the Nyquist bin.

`transform_positive_lag_correlation`

```
def transform_positive_lag_correlation(
    correlation: ArrayLike,
    *,
    dt_ps: float,
    n_fft: int,
    tensor_axes: tuple[int, int] | None = None,
) -> tuple[FloatArray, ComplexArray]:
    ...
```

Inputs

- `dt_ps` is finite and strictly positive;
- `n_fft` satisfies the reconstruction bound;
- the correlation follows the layout required by `reconstruct_two_sided_correlation`.

Algorithm

1. Reconstruct the real two-sided work array.

2. Apply `scipy.fft.rfft(..., axis=0)`.
3. Multiply by Δt.
4. Apply the one-sided density scale along the frequency axis.
5. Return `scipy.fft.rfftfreq(n_fft, d=dt_ps)` and the complex spectrum.

Output units

If the input correlation has units U, the output density has units

$$U \cdot \mathrm{ps}.$$

The helper does not assign a textual unit. The public caller does so from the physical correlation metadata.

Required invariants

- frequency is axis zero;
- the frequency grid starts at zero and is uniformly spaced;
- a scalar or diagonal autocorrelation produces a real spectrum up to floating-point roundoff;
- a tensor reconstructed with `tensor_axes` produces a Hermitian matrix at every frequency;
- for an applicable scalar one-sided density,

$$\Delta f \sum_m P_+(f_m) = C(0)$$

within floating-point tolerance; - zero padding does not change this discrete bin measure.

Failure policy

The module raises `TypeError` for invalid scalar types and `ValueError` for invalid values, shapes, axes, lengths, or nonfinite arrays. It does not issue scientific warnings and does not clip spectra.

Test plan

The focused test module `tests/test_spectral.py` covers:

- lower-bound and padding resolution;
- rectangular, half-Hann, half-Tukey, and custom windows;
- scalar and tensor reconstruction;
- odd/even one-sided scale vectors;
- direct $O(LF)$ DFT agreement;
- discrete spectral-area identity;
- tensor Hermiticity;
- invalid dimensions, axes, values, and parameters.

The direct DFT oracle is intentionally independent of the production `rfft` path.

N1.6 discrete spectral-bin integration

Function

```
def spectral_bin_integral(
    spectrum: ArrayLike,
    frequencies_thz: ArrayLike,
    *,
    axis: int = 0,
) -> NDArray[np.float64] | np.float64:
    ...
```

Motive

A one-sided FFT density is stored as values associated with uniform frequency bins. Its normative total weight is therefore

$$I = \Delta f \sum_m P_m,$$

not a trapezoidal approximation to a separately interpolated curve. Applying trapezoidal endpoint half-weights would count the one-sided DC and Nyquist bookkeeping a second time and would break the discrete identity used by VS1.

This bin measure is an mdstats numerical convention built on the one-sided FFT contract already specified above. It is not presented as a new quadrature algorithm.

Inputs and constraints

- `spectrum` is a finite, real array with at least one dimension;
- `frequencies_thz` is finite, one-dimensional, and contains at least two samples;
- the frequency count equals `spectrum.shape[axis]`;
- frequencies are strictly increasing and uniformly spaced;
- a cropped grid may start above zero, provided its original FFT spacing is preserved;
- `axis` is an integer and may be negative;
- complex arrays are rejected rather than silently discarding an imaginary part.

Algorithm

```
1. Convert the real spectrum to float64 without modifying the input.
2. Normalize and validate the frequency axis.
3. Validate finite, strictly increasing, uniform frequencies.
4. Resolve df from the adjacent-bin increments.
5. Return df * sum(spectrum, axis=axis, dtype=float64).
```

Output

For a one-dimensional spectrum the output is a NumPy scalar. For additional projection axes, the output has the same shape as `spectrum` with the frequency axis removed.

Complexity

For M stored values, time complexity is $O(M)$. No interpolation grid or cumulative work array is allocated.

Required tests

- scalar and multidimensional bin sums;
- positive and negative frequency-axis indices;
- a uniformly cropped nonzero-start grid;
- zero-padding invariance when applied to VS1 spectra;
- deliberate difference from trapezoidal endpoint weighting;
- rejection of nonuniform, repeated, decreasing, nonfinite, complex, or shape-incompatible inputs.

Deferred functions

The following roadmap functions remain outside this stage:

- Welch segment periodograms;
- atom-spectrum memory planning.

VDOS normalization is specified separately in `vdos_spec.md` and consumes `spectral_bin_integral`.

References

[1] N. Wiener, "Generalized Harmonic Analysis," *Acta Mathematica* **55**, 117-258 (1930). DOI: 10.1007/BF02546511.

[2] A. Khintchine, "Korrelationstheorie der stationaren stochastischen Prozesse," *Mathematische Annalen* **109**, 604-615 (1934). DOI: 10.1007/BF01449156.

[3] F. J. Harris, "On the Use of Windows for Harmonic Analysis with the Discrete Fourier Transform," *Proceedings of the IEEE* **66**, 51-83 (1978). DOI: 10.1109/PROC.1978.10837.

[4] P. Virtanen, R. Gommers, T. E. Oliphant, et al., “SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python,” *Nature Methods* **17**, 261-272 (2020). DOI: 10.1038/s41592-019-0686-2.